ESPIGA

CARPETA DEL PROYECTO · DOCUMENTACIÓN

Espiga · Sistema de gestión para PCR Amoblamientos

Documentación técnica, operativa y económica del proyecto. Sistema web para gestionar clientes con cuenta corriente automática, proveedores, pedidos, archivos técnicos, cobranzas y entregas de un taller de muebles a medida, publicado en la nube y accesible desde cualquier dispositivo.

Cliente: Rodrigo Pasut · PCR Amoblamientos, Villa Bosch

Materia: Evaluación de Proyectos

Ciclo: 7mo Informática 2026 · Instituto Leonardo Murialdo

Equipo: Felipe Araujo · Valentina Autelli · Mía Yorio

CONTENIDOS

1. Control de Cambios
2. Diagnóstico: Árbol de Problemas
3. Introducción
4. Descripción General
5. Requisitos Específicos
6. Modelo Entidad-Relación (3FN)
7. Diagrama de flujo
8. Oferta de Gestión
9. Oferta Económica
10. Estimación Económica
11. Contrato
12. Bitácora del Proyecto
13. Guía de Inicio Rápido
14. Preguntas Frecuentes

Sobre este documento. Sigue la estructura de la **Carpeta del Proyecto** del módulo de Documentación de la materia. Los apartados propios de proyectos con hardware (interfaz de hardware, visualización 3D, técnico instalador) fueron retirados con su aclaración, porque Espiga es un sistema 100% software. El modelo de datos se documenta normalizado hasta 3FN, igual que exige el módulo.

1. Control de Cambios

Registro de las modificaciones sobre la documentación y el alcance del proyecto, con fecha, responsable y detalle. Cada validación con Rodrigo que derive en un cambio de requerimiento se anota como una fila nueva.

Versión	Fecha	Responsable	Detalle del cambio
0.1	10/05/2026	Valentina Autelli · Mía Yorio	Versión inicial a partir del documento de análisis (mockup, roles, diagrama entidad-relación y diagrama de actividades).
0.2	28/08/2026	Felipe Araujo	Arquitectura de 3 perfiles (administrador, administrativo, operario) y cobertura completa de RF01 a RF11.
0.3	04/09/2026	Felipe Araujo	Se quitaron los usuarios y claves de ejemplo de la pantalla de login, por ser un hueco de seguridad detectado durante el desarrollo.
0.4	04/09/2026	Felipe Araujo	Alta de recuperación de contraseña por email (RF09). Queda pausada hasta obtener el email real del administrador.
0.5	07/09/2026	Felipe Araujo	Publicación del sistema en Railway y alta de subida real de archivos (RF07) con almacenamiento persistente.
0.6	11/09/2026	Felipe Araujo	Cambio de dominio público a `espiga-pcr.up.railway.app` por uno más prolijo para presentar al cliente.
0.7	18/09/2026	Felipe Araujo	Alta de RF12 a RF17: comprobantes al cliente, resumen de cuenta en PDF, consumo de materiales, buscador y filtros, app instalable (PWA) y notificaciones push de alertas.
0.8	25/09/2026	Felipe Araujo	Alta de RF18 a RF22: backup automático, presupuesto en PDF, estadísticas, calendario de entregas y recordatorios de cobranza. Modelo de datos normalizado a 3FN.
0.9	02/10/2026	Felipe Araujo	Rediseño estético de toda la interfaz manteniendo la identidad madera del taller.
1.0	02/10/2026	Felipe Araujo	Alta de RF23 a RF33: seguimiento del pedido para el cliente, presupuesto por ítems con QR, Excel, historial de estados, aviso push al cliente, stock de insumos, tablero Kanban, galería de fotos, dashboard, flujo de caja, backup por mail y modo oscuro. Se activa la recuperación de clave por email (RF09). Modelo de datos: 16 tablas.
1.1	05/10/2026	Felipe Araujo	Alta de RF34 a RF37, pedidos por el cliente: varias cuentas por proveedor con saldo propio, flete aparte del presupuesto, entrega en días hábiles con feriados y saldos de proveedores por separado en el panel. Modelo de datos: 18 tablas.
1.2	21/10/2026	Felipe Araujo	Alta de RF38: puesta en marcha con datos reales (se quitan los datos de ejemplo, con backup previo). Se actualizan la Guía de inicio rápido y las Preguntas frecuentes.
1.3	05/11/2026	Felipe Araujo	Documentación al día y publicada en PDF; identidad corporativa y pieza visual para la Expo; responsables, fechas y costos cargados.

2. Diagnóstico: Árbol de Problemas y Soluciones

Diagnóstico elaborado por el equipo a partir del relevamiento en PCR Amoblamientos. Es la pieza que fundamenta todas las decisiones de diseño: el problema central y sus causas explican por qué el sistema gestiona lo que gestiona, y los medios del árbol de soluciones son el origen directo de los requerimientos funcionales.

Cómo se lee el árbol

El árbol de problemas se lee de abajo hacia arriba: las causas producen el problema central, y el problema central produce los efectos. El árbol de soluciones es su espejo: cada causa se convierte en un medio y cada efecto en un fin medible.

Causas → producen → Problema central → produce → Efectos

Problema central

PCR Amoblamientos **no tiene una forma centralizada ni confiable de saber cuánto le debe cada cliente, cuánto le debe a cada proveedor y en qué estado está cada pedido.** La gestión es 100% manual (papel, cuadernos y memoria): la información se dispersa, se pierde y no hay trazabilidad.

Causas

Condiciones que producen el problema central. Cada una se relevó con Rodrigo y se traduce después en un requerimiento concreto.

Causa	Qué se observó en el relevamiento
El saldo de cada cliente se calcula a mano	Cobros y entregas parciales se anotan en distintos lugares; el saldo real depende de sumar a mano varios papeles, y se presta a errores.
No hay un registro único de pedidos	Los pedidos viven en la cabeza de Rodrigo y en anotaciones sueltas; no hay una lista con el estado de cada uno.
Las deudas con proveedores no están consolidadas	Las facturas de madereras y herrajerías se acumulan sin un total claro de cuánto se debe a cada proveedor.
No hay aviso de vencimiento de entregas	Un pedido puede pasarse de la fecha comprometida sin que nadie lo note a tiempo.
Los planos y listas de corte se traspapelan	Los archivos técnicos de cada trabajo (planos, listas de corte) están en distintas compus, mails y papeles; cuesta encontrarlos.
No hay registro histórico ni trazabilidad	No se puede reconstruir quién cargó un pago, ni comparar un mes con otro, ni saber qué cliente cobra más peso en el negocio.

Efectos

Consecuencias que el problema produce hoy en el taller. Sus valores actuales forman la línea de base contra la que se mide el proyecto.

Efecto	Cómo se manifiesta hoy (línea de base)
Saldos poco confiables	El saldo de un cliente puede tardar minutos en reconstruirse y no siempre coincide con la realidad.
Cobranzas que se pierden	Clientes con saldo pendiente que no se reclaman a tiempo porque nadie tiene la lista a mano.
Entregas que se atrasan sin aviso previo	El taller se entera de un atraso cuando el cliente pregunta, no antes.
Tiempo perdido buscando información	Encontrar un plano, un pago o el estado de un pedido implica revisar papeles y varias compus.
Decisiones sin datos	No se sabe qué cliente pesa más, cuánto se cobró el mes pasado, ni a qué proveedor se le compra más.

Árbol de soluciones

Objetivo central: darle a PCR Amoblamientos una **gestión centralizada y confiable** de clientes, pedidos, pagos, proveedores y entregas, con saldos calculados solos y toda la información en un único lugar accesible desde cualquier dispositivo.

Cada **medio** neutraliza una causa identificada más arriba y se convierte después en uno o varios requerimientos.

Medio	Causa que neutraliza
Cuenta corriente con saldo automático. El saldo se deriva de los pagos, nunca se guarda a mano.	El saldo de cada cliente se calcula a mano.
Registro único de pedidos con estado. Todos los pedidos en una lista, con su estado de producción y entrega.	No hay un registro único de pedidos.
Gestión de proveedores con deuda consolidada. Cada factura recibida suma a un total claro por proveedor.	Las deudas con proveedores no están consolidadas.
Alertas de entrega + notificación al celular. Aviso automático cuando una fecha se acerca.	No hay aviso de vencimiento de entregas.
Archivos por pedido con almacenamiento persistente. Planos y listas de corte guardados junto al pedido.	Los planos y listas de corte se traspapelan.
Base de datos histórica + estadísticas + backup. Cada movimiento queda registrado y respaldado.	No hay registro histórico ni trazabilidad.

Los **fines** son el espejo de los efectos: expresan a qué valor debería llegar cada uno con el sistema funcionando.

Fin	Meta verificable	Punto de partida
Saldo confiable e instantáneo	El saldo de cualquier cliente se ve al instante y siempre coincide con los pagos cargados.	Minutos, sumando papeles
No perder cobranzas	Lista de clientes con saldo siempre disponible, con recordatorio listo para enviar.	Se pierden por olvido
Detectar atrasos a tiempo	Aviso automático 7 días antes de la fecha de entrega, en el sistema y en el celular.	Cuando el cliente pregunta
Información en un solo lugar	Cliente, pedido, pago o plano encontrados en segundos desde cualquier dispositivo.	Papeles y varias compus

Del árbol al resto del documento

Este diagnóstico alimenta la **línea de base** y los **OKRs**: los efectos aportan los valores de partida y los fines aportan los Key Results contra los que se evalúa el proyecto. Lo que el árbol identifica pero el sistema no resuelve en esta entrega queda declarado en las Limitaciones del alcance (5.1), y las mejoras diferidas, en la Evolución previsible (4.6).

Objetivo 1 — Centralizar y hacer confiable la información del taller

- **KR 1.1** — El saldo de cualquier cliente se calcula y muestra automáticamente, sin sumas manuales.
- **KR 1.2** — El 100% de los pedidos activos tienen su estado y fecha de entrega registrados en el sistema.
- **KR 1.3** — Todo pago queda registrado con el usuario que lo cargó (trazabilidad).

Objetivo 2 — No perder cobranzas ni entregas

- **KR 2.1** — El sistema avisa de toda entrega que esté a 7 días o menos de su fecha.
- **KR 2.2** — Existe en todo momento una lista de clientes con saldo pendiente, con el mensaje de cobranza listo para enviar.

Objetivo 3 — Que el sistema no dependa de un equipo del taller

- **KR 3.1** — El sistema es accesible desde cualquier dispositivo con internet, sin instalar nada.
- **KR 3.2** — Los datos se respaldan automáticamente todos los días, con retención de los últimos 7 backups.

3. Introducción

3.1. Propósito

Dotar a PCR Amoblamientos de una forma **centralizada y confiable** de gestionar su operación diaria: clientes con cuenta corriente, pedidos, pagos, proveedores, archivos técnicos y entregas. Hoy toda esa gestión es manual y dispersa, lo que genera saldos poco confiables, cobranzas que se pierden y entregas que se atrasan sin aviso.

Espiga busca resolver tres problemas medibles: la **falta de un saldo confiable e inmediato** por cliente, la **pérdida de cobranzas** por no tener la información a mano, y la **detección tardía de atrasos** en las entregas.

3.2. Alcance

El sistema **incluye**: inicio de sesión con 3 perfiles, panel de saldos, gestión de clientes con cuenta corriente automática, gestión de proveedores con deudas, gestión de pedidos con estados y archivos técnicos, registro de consumo de materiales, emisión de comprobantes (Factura A / B / Recibo), resumen de cuenta y presupuesto en PDF, alertas de entrega (en el sistema y por notificación al celular), calendario de entregas, estadísticas del negocio, recordatorios de cobranza, buscador y filtros, gestión de usuarios, backup automático y la posibilidad de instalar el sistema como app en el celular. Todo publicado en la nube.

El sistema **no incluye**: factura electrónica ni integración con AFIP (sí emite comprobantes simples), control de stock de materiales más allá del consumo por pedido, envío 100% automático de recordatorios sin intervención, ni una app nativa de tienda (se instala como PWA). El detalle está en las Limitaciones (5.1) y se formaliza en el Contrato (sección 9).

3.3. Personal Involucrado

Roles del equipo de proyecto, según los Lineamientos 2026 (PM, desarrollo Frontend/Backend, UX/UI, base de datos). Al ser un proyecto 100% software, no incluye el rol de técnico instalador propio de los proyectos con hardware.

Rol	Responsabilidad principal	Integrante
Project Manager (PM)	Planificación temporal (GANTT), control de alcance y relación con el cliente.	Valentina Autelli
Backend / BD	Servidor Node.js + Express, base de datos MySQL, API REST, despliegue.	Felipe Araujo
Frontend / UX-UI	Interfaz web, mockup y validación de la interfaz con el usuario.	Felipe Araujo
Documentación / QA	Documentación técnica y de usuario, pruebas y control de calidad.	Mía Yorio
Cliente	Valida requerimientos, prioriza y firma el contrato.	Rodrigo Pasut (PCR Amoblamientos)

3.4. Definiciones, acrónimos y abreviaturas

Término	Definición
ABM	Alta, Baja y Modificación: operaciones básicas de gestión de un registro (cliente, proveedor, usuario).
Cuenta corriente	Registro de los movimientos (presupuesto y pagos) de un cliente, del que se deriva el saldo.
Saldo	Diferencia entre lo presupuestado y lo pagado, calculada por el sistema (nunca guardada a mano).
API REST	Interfaz HTTP que expone los datos del backend al frontend, en formato JSON.
Backend	Servidor (Node.js + Express) que procesa la lógica y se conecta a la base de datos.
Frontend	Interfaz web que consume la API REST y que usan Rodrigo, Ana y Facu.
Vista (SQL)	Consulta guardada que calcula datos derivados (como el saldo) en el momento, sin almacenarlos.
Trigger	Regla automática en la base de datos; en Espiga, cierra el pedido cuando está entregado y saldado.
JWT	Token de sesión que identifica al usuario logueado en cada pedido a la API.
PWA	Progressive Web App: web instalable en el celular sin pasar por una tienda de aplicaciones.
Push (VAPID)	Notificación que llega al celular aunque la app esté cerrada. VAPID son las claves que la autorizan.
RF / RNF	Requerimiento Funcional / Requerimiento No Funcional.
ISW	Interfaz de Software: componente de software del sistema y su tecnología.
3FN	Tercera Forma Normal: nivel de normalización que evita que un dato quede repetido y se contradiga.
OKR / KR	Objective and Key Results / Resultado Clave (métrica de progreso).
Línea de base	Medición de la situación inicial (antes del sistema) usada como referencia de comparación.

3.5. Referencias

Formato APA 7ª edición.

- Pedaci, L. (2026). *Evaluación de Proyectos - Módulo 01: El Proyecto y sus generalidades*. Instituto Leonardo Murialdo.
- Pedaci, L. (2026). *Evaluación de Proyectos - Módulo 02: Fases de un Proyecto*. Instituto Leonardo Murialdo.
- Pedaci, L. (2026). *Evaluación de Proyectos - Módulo de Documentación*. Instituto Leonardo Murialdo.
- Pedaci, L. (2026). *Lineamientos y Metodologías de Trabajo - Proyecto Tecnológico Interdisciplinario*. Instituto Leonardo Murialdo.
- Universidad Tecnológica Nacional. (2012). *Buenas Prácticas para Proyectos Informáticos*. edUTecNe. ISBN 978-987-1896-01-1.
- Institute of Electrical and Electronics Engineers. (1998). *IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications*.

3.6. Resumen Ejecutivo

Espiga es un sistema de gestión web para PCR Amoblamientos, un taller de muebles a medida ubicado en Villa Bosch. Centraliza clientes con cuenta corriente automática, proveedores con seguimiento de deudas, pedidos con alertas de entrega y archivos técnicos (planos, fotos, listas de corte) por pedido, con tres perfiles de acceso: administrador (Rodrigo), administrativo (Ana) y operario (Facu). Está construido con backend Node.js + Express y base de datos MySQL, y publicado en Railway para usarse desde cualquier dispositivo sin depender de una computadora del taller.

Público objetivo: talleres y pequeños fabricantes a medida que llevan clientes, proveedores y pedidos de forma manual. **Resultados esperados:** pasar de un saldo que se reconstruye a mano y a veces no coincide, a un saldo confiable e instantáneo; no volver a perder cobranzas por olvido; y detectar los atrasos de entrega 7 días antes en lugar de cuando el cliente pregunta.

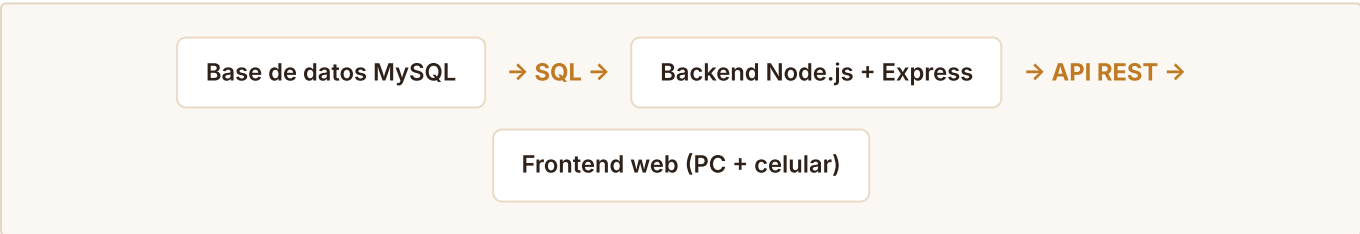
---

4. Descripción General

4.1. Perspectiva del producto

Espiga es un **sistema nuevo**: hoy el taller no tiene ningún sistema de gestión, todo es manual (papel, cuadernos, memoria). No reemplaza ni mejora un software existente. Se apoya en infraestructura que el taller ya tiene (una PC y celulares con conexión a internet) y se publica en la nube para no depender de ningún equipo encendido en el local.

Arquitectura en tres capas (persistencia, lógica de negocio y presentación):



4.2. Funcionalidad del producto

A nivel general (sin entrar todavía en los requisitos del 5.1), el producto permite:

- Iniciar sesión según el rol de cada persona del taller (administrador, administrativo, operario).
- Ver de un vistazo el saldo de todos los clientes y proveedores en un panel único.
- Registrar clientes y llevar su cuenta corriente, con el saldo calculado automáticamente.
- Registrar proveedores y llevar el seguimiento de las deudas del taller con cada uno.
- Cargar pedidos, adjuntarles planos, fotos y listas de corte, registrar el consumo de materiales y seguir su estado.
- Emitir comprobantes (Factura A / B / Recibo) y generar presupuestos y resúmenes de cuenta en PDF.
- Recibir alertas cuando una entrega se acerca, en el sistema y por notificación al celular.
- Ver los pedidos por fecha de entrega en un calendario, consultar estadísticas del negocio y generar recordatorios de cobranza.
- Buscar y filtrar, gestionar usuarios, respaldar los datos automáticamente e instalar el sistema como app en el celular.

4.3. Características de los Usuarios Buenas Prácticas UTN p.56/57

Siguiendo el formato de la tabla de usuarios del libro UTN (tipo de usuario, formación y actividades). El sistema define tres perfiles de acceso, alineados con la matriz de permisos implementada en el backend.

Administrador — Rodrigo

Dueño del taller, 40+ años, comodidad tecnológica media. Acceso total: ve todos los módulos, crea pedidos, gestiona usuarios, revisa backups y estadísticas, y podrá recuperar su clave por email una vez configurada esa función. Uso diario.

Administrativa — Ana

Lleva clientes, proveedores, cobranzas y facturación. Acceso a cuentas corrientes, pagos, comprobantes, alertas, calendario, estadísticas y recordatorios; no crea pedidos ni gestiona usuarios. Uso diario.

Operario — Facu

Trabaja del lado de producción: consulta y actualiza el estado de los pedidos, carga materiales y sube archivos (planos, listas de corte). No accede a saldos, cobranzas ni gestión de usuarios. Uso ocasional durante la jornada.

4.4. Restricciones Buenas Prácticas UTN p.57

4.4.1. Políticas reguladoras

Se desarrolla con software libre (Node.js, Express, MySQL). No hay costo de licencia. El único costo externo posible es el alojamiento en la nube (4.4.2). El repositorio se aloja en GitHub (`felooaraujo-ai/Kore`); si es privado, se incluye al equipo docente como colaborador.

4.4.2. Limitaciones de infraestructura

El backend necesita estar disponible de forma continua para acceder desde el taller o desde afuera. Se resolvió con Railway, con un volumen persistente (`kore-volume` , montado en `/app/uploads`) para que los archivos subidos y los backups no se pierdan entre despliegues.

4.4.3. Interfaces con otras aplicaciones

El sistema no se integra, en esta entrega, con AFIP, software contable ni pasarelas de pago. Es autónomo. Para las notificaciones push usa el estándar Web Push (con claves VAPID propias, sin depender de un servicio pago).

4.4.4. Función de control

Autenticación con usuario y contraseña, y control de acceso por rol (administrador / administrativo / operario). La pantalla de login no muestra usuarios ni claves de ejemplo. Cada pago y cada comprobante queda registrado con el usuario que lo cargó (trazabilidad).

4.4.5. Requisitos del lenguaje

Toda la interfaz está en español, con los términos que usa el taller (cliente, proveedor, pedido, saldo) en lugar de jerga de sistemas.

4.4.6. Protocolos señalados

Comunicación frontend → backend por HTTP/REST (JSON) con token JWT. Backend → base de datos por SQL sobre MySQL. Notificaciones por Web Push. Recuperación de clave por SMTP (email).

4.4.7. Requisitos de fiabilidad

El sistema debe estar disponible durante el horario del taller y no perder archivos entre reinicios, gracias al volumen persistente. Un backup automático diario protege los datos, con retención de los últimos 7 respaldos.

4.4.8. Consideraciones de seguridad

Contraseñas almacenadas con hash (bcrypt), nunca en texto plano; sesiones con token JWT; acceso restringido por rol; sin datos de ejemplo en el login. La base de datos no tiene acceso público (solo el backend le habla por la red interna). La recuperación de clave por email está desarrollada, pausada hasta contar con el email real del administrador.

4.5. Suposiciones y dependencias

- Se asume que el taller cuenta con conexión a internet para acceder al sistema publicado en Railway.
- El proyecto **depende** de que Rodrigo confirme su email real para activar la recuperación de contraseña (RF09).
- Las notificaciones push en iPhone requieren que el sistema esté instalado como app en la pantalla de inicio (restricción de Apple, no del sistema).
- Se asume que los tres roles (administrador, administrativo, operario) cubren las necesidades del taller sin variantes adicionales.

4.6. Evolución previsible del sistema

Mejoras identificadas para versiones futuras (surgen de las Limitaciones del 5.1):

- **v2** — Activar la recuperación de contraseña por email, una vez confirmado el email real de Rodrigo.
- **v2** — Envío de recordatorios y alertas de forma totalmente automática (email o WhatsApp), sin intervención; hoy el push al celular ya funciona y los recordatorios se generan listos para enviar.
- **v2** — Ampliar reportes PDF y estadísticas (ya existen presupuesto, resumen de cuenta y un panel de estadísticas base).
- **v3** — Control de stock de materiales, más allá del registro de consumo por pedido.
- **v3** — Facturación electrónica con integración a AFIP, si el taller lo requiere.

---

5. Requisitos Específicos

Apartados retirados del modelo de referencia: las «Interfaces de hardware» y la «Visualización 3D del producto» no aplican: Espiga es un sistema 100% software, sin ningún componente físico que diseñar o fabricar. El «Técnico instalador» tampoco existe como rol.

5.1. Requerimientos funcionales Buenas Prácticas UTN p.59/60

Acciones concretas que el sistema debe poder realizar. Cada requerimiento indica su prioridad y su origen (el síntoma o la necesidad del que surge).

ID	Requerimiento	Prioridad	Origen
RF01	**Inicio de sesión con 3 roles.** Login sin usuarios ni claves de ejemplo visibles, con acceso diferenciado por rol.	Alta	Cada rol ve solo lo suyo
RF02	**Panel de saldos.** Vista única con el estado de las cuentas corrientes de clientes y proveedores.	Alta	"No sé cuánto me deben en total"
RF03	**ABM de clientes con cuenta corriente automática.** El saldo se recalcula solo con cada pago.	Alta	El saldo se calcula a mano
RF04	**ABM de proveedores con gestión de deudas.** Seguimiento de lo que el taller debe a cada uno.	Alta	Deudas no consolidadas
RF05	**Gestión de pedidos.** Alta y seguimiento de pedidos, con su estado de producción y entrega.	Alta	No hay registro único de pedidos
RF06	**Alertas de entrega.** Aviso cuando la fecha comprometida de un pedido se acerca o se vence.	Alta	Atrasos sin aviso previo
RF07	**Carga y almacenamiento de archivos por pedido.** Planos, fotos y listas de corte guardados de forma persistente.	Alta	Los planos se traspapelan
RF08	**Gestión de usuarios.** Alta, edición y asignación de rol para cada persona del taller.	Media	Control de acceso por rol
RF09	**Recuperación de contraseña por email.** Envío de un link para poner una clave nueva. Activa mediante la API de Brevo (envío por HTTPS).	Media	"¿Y si me olvido la clave?"
RF10	**Sección de ayuda «¿Cómo funciona?».** Guía dentro del propio sistema para los tres roles.	Baja	Onboarding sin capacitación
RF11	**Publicación online del sistema.** Acceso desde cualquier dispositivo, sin depender de una compu del taller.	Alta	No depender de un equipo local
RF12	**Emisión de comprobantes al cliente.** Factura A, Factura B o Recibo por un pedido, con número, fecha y monto.	Media	Comprobante al cobrar
RF13	**Resumen de cuenta en PDF.** Detalle de pedidos, pagos y saldo de un cliente, exportable.	Media	Rendir cuenta al cliente
RF14	**Registro de consumo de materiales.** Materiales usados por pedido (descripción y cantidad).	Media	Saber qué se usó en cada trabajo
RF15	**Buscador y filtros.** Búsqueda por texto y filtro por período (Hoy / Esta semana / Todos) en Clientes y Pedidos.	Media	Encontrar rápido
RF16	**App instalable en el celular.** Se agrega a la pantalla de inicio (Android/iPhone) y abre a pantalla completa.	Media	Acceso rápido en la calle

ID	Requerimiento	Prioridad	Origen
RF17	**Notificaciones push de alertas.** Aviso automático al celular cuando un pedido está a 7 días o menos de su entrega.	Media	Enterarse sin entrar al sistema
RF18	**Backup automático de datos.** Copia diaria de todos los datos, descarga manual y retención de los últimos 7.	Media	No perder datos
RF19	**Presupuesto en PDF para el cliente.** Presupuesto formal por pedido (validez, seña sugerida, condiciones) para enviar antes de confirmar.	Media	Cerrar trabajos nuevos
RF20	**Estadísticas del negocio.** Mejores clientes, cobrado por mes y proveedores con más compras, en vivo.	Baja	Decidir con datos
RF21	**Vista calendario de entregas.** Pedidos por fecha en un calendario mensual, con color por urgencia y filtros.	Media	Planificar la producción
RF22	**Recordatorios a clientes con saldo.** Lista de deudores con un mensaje listo para copiar o abrir en WhatsApp.	Media	No perder cobranzas
RF23	**Seguimiento del pedido para el cliente.** Link público sin login (código único) con estado, entrega estimada y pago.	Media	"¿Cómo va lo mío?"
RF24	**Presupuesto por ítems.** Renglones con cantidad y precio; el total se calcula solo.	Media	Presupuestar con detalle
RF25	**Código QR en el presupuesto.** El PDF incluye un QR al seguimiento del cliente.	Baja	Enlace rápido
RF26	**Exportación a Excel.** Clientes, pedidos y pagos en .xlsx.	Media	Llevar a la planilla
RF27	**Historial de estados del pedido.** Línea de tiempo con fecha y responsable.	Baja	Trazabilidad
RF28	**Aviso push al cliente.** Notificación cuando el pedido cambia de estado.	Baja	Cliente informado
RF29	**Stock de insumos.** Inventario con alerta de bajo stock; el consumo descuenta.	Media	Saber qué reponer
RF30	**Tablero Kanban de producción.** Se arrastran los pedidos entre etapas.	Media	Ver el flujo
RF31	**Galería de fotos del mueble.** Visibles también en el seguimiento.	Baja	Mostrar el avance
RF32	**Dashboard con gráficos.** Cobrado por mes, pedidos por estado y mejores clientes.	Baja	Decidir con datos
RF33	**Flujo de caja, backup por mail y modo oscuro.** Entradas vs. salidas, copia por email y tema oscuro.	Baja	Control y comodidad
RF34	**Cuentas por proveedor.** Varias cuentas por proveedor, cada una con su saldo (le debés / a favor) y sus movimientos de compras y pagos.	Media	Separar compras del mismo proveedor

ID	Requerimiento	Prioridad	Origen
RF35	**Flete aparte del presupuesto.** Se carga el día de la entrega y se suma al total a cobrar como línea separada.	Media	Cobrar el flete sin mezclarlo
RF36	**Entrega en días hábiles.** Calcula la fecha salteando fines de semana y feriados nacionales; avisa desde 7 días antes y el mismo día.	Media	Plazos de 45 a 65 días hábiles
RF37	**Saldos de proveedores por separado.** El panel muestra cada proveedor con su saldo, sin sumarlos en un total.	Baja	No mezclar cuentas
RF38	**Puesta en marcha.** El administrador quita los datos de ejemplo de fábrica, con un backup previo, sin tocar lo cargado desde la app.	Baja	Arrancar con datos reales

Requerimientos no funcionales (RNF)

ID	Requerimiento	Categoría
RNF01	Contraseñas almacenadas con hash (bcrypt), nunca en texto plano.	Seguridad
RNF02	La pantalla de login no muestra usuarios ni claves de ejemplo.	Seguridad
RNF03	Accesible desde PC y celular por navegador, sin instalar nada; opcionalmente instalable como app (RF16).	Usabilidad
RNF04	Interfaz completa en español, con el vocabulario que ya usa el taller.	Usabilidad
RNF05	Los archivos subidos y los backups se conservan entre reinicios del servicio (volumen persistente).	Fiabilidad
RNF06	Comunicación frontend-backend por API REST sobre HTTP, autenticada con JWT.	Protocolos
RNF07	Los saldos nunca se almacenan: se calculan en el momento con vistas SQL, para que no puedan desincronizarse.	Integridad de datos

Limitaciones del alcance

Definen qué **no** hace el sistema en esta entrega. No son debilidades: son un contrato de honestidad con el usuario. Cada limitación puede convertirse en una versión futura (4.6).

ID	Qué NO hace en esta entrega	Por qué	¿Versión futura?
LIM01	No emite factura electrónica ni se integra con AFIP (sí emite comprobantes simples, RF12).	La factura electrónica requiere homologación con AFIP, fuera del alcance académico.	Sí. v3.
LIM02	~~No lleva stock de materiales.~~ **Implementado (RF29):** stock de insumos con alerta de bajo stock y descuento por consumo.	Se sumó como módulo propio en la Fase 5.	Hecho (v2).

ID	Qué NO hace en esta entrega	Por qué	¿Versión futura?
LIM03	No hay app móvil nativa de tienda; se instala como PWA desde el navegador (RF16).	Una app nativa duplica el desarrollo y requiere cuentas de tienda pagas.	A evaluar.
LIM04	~~La recuperación de contraseña por email no está activa.~~ **Activa (RF09)** mediante la API de Brevo.	Railway bloquea el SMTP saliente; se resolvió con la API HTTP de Brevo.	Hecho (v2).
LIM05	Los recordatorios de cobranza se envían de forma asistida (mensaje listo para WhatsApp); el backup y la recuperación de clave sí salen por email automático.	El envío automático a WhatsApp requiere una API de terceros paga.	Sí. v3.

5.2. Interfaces de usuario Buenas Prácticas UTN p.58

El único punto de contacto de Rodrigo, Ana y Facu con el sistema es el **frontend web**, diseñado con criterio de usabilidad para un usuario de comodidad tecnológica media. En escritorio tiene una barra lateral de navegación (Panel de saldos, Clientes, Proveedores, Pedidos, Calendario, Alertas, Estadísticas, Recordatorios, Usuarios, Backups, ¿Cómo funciona?) y en celular una barra inferior simplificada (Inicio, Clientes, Prov., Pedidos, Alertas). Clientes y Pedidos tienen buscador y filtro por período. El login pide usuario y clave, sin datos de ejemplo, e incluye «¿Olvidaste tu clave?». El sistema se puede instalar como app (RF16) y activar notificaciones push (RF17). La estética sigue una identidad "madera" coherente con un taller de muebles.

5.2.1. Diseño de interfaz

Antes de conectar la lógica, el equipo trabajó sobre un mockup navegable y lo fue validando; el feedback se registra como hito en el control de cambios y en el Registro de Cambios de Interfaz (10.3).

5.3. Interfaces de software Buenas Prácticas UTN p.59

Componentes de software del sistema y cómo se comunican. No es lo mismo que los RNF (rendimiento y protocolos) ni que los RF (funcionalidades).

ID	Componente	Tecnología / Stack	Rol en el sistema
ISW01	**Frontend**	HTML, CSS y JavaScript, conectado a la API real.	Interfaz que usan Rodrigo, Ana y Facu según su rol.
ISW02	**Backend / API REST**	Node.js + Express. Auth: JWT.	Lógica de negocio: cuenta corriente, alertas, permisos por rol.
ISW03	**Base de datos**	MySQL, con vistas y triggers.	Persistencia central. Los saldos se derivan con vistas. Modelo en 5.6.
ISW04	**Almacenamiento de archivos**	Volumen persistente de Railway (`kore-volume` , `/app/uploads`).	Guarda planos, fotos, listas de corte y los backups.
ISW05	**Hosting / despliegue**	Railway, con despliegue automático desde GitHub.	Publica backend y frontend en `espiga-pcr.up.railway.app` .

ID	Componente	Tecnología / Stack	Rol en el sistema
ISW06	**Recuperación de contraseña**	Email (Nodemailer/SMTP), desarrollado y pausado.	Implementa RF09, a la espera del email real del administrador.
ISW07	**Notificaciones push**	Web Push API + claves VAPID + Service Worker.	Implementa RF17: avisa al celular sin depender de un tercero pago.
ISW08	**Generador de PDF**	PDFKit (Node.js), del lado del servidor.	Implementa RF13 y RF19: resumen de cuenta y presupuesto.
ISW09	**Backup automático**	Tarea programada en el backend; vuelca los datos a un archivo SQL en el volumen.	Implementa RF18: copia diaria, retención de 7, descarga por el administrador.

5.4. Interfaces de comunicación Buenas Prácticas UTN p.59

Interfaz	Protocolo / Formato	Detalle
Frontend → Backend	HTTP / REST (JSON)	Endpoints autenticados con JWT para clientes, proveedores, pedidos, pagos, alertas y usuarios.
Backend → Base de datos	SQL (MySQL)	Consultas y actualizaciones sobre el esquema relacional (5.6). Puerto 3306, red interna.
Backend → Almacenamiento	Sistema de archivos (volumen)	Lectura y escritura de los archivos subidos y de los backups, por pedido.
Backend → Navegador (push)	Web Push (VAPID)	Envío de las notificaciones de alerta de entrega al Service Worker del celular.
Recuperación → Usuario	SMTP (email)	Envío del link para poner una clave nueva. Pendiente de activar (LIM04).

5.6. Modelo Entidad-Relación de la base de datos

Modelo de datos relacional que sostiene la persistencia del sistema (componente ISW03). Representa qué guarda Espiga y cómo se relacionan las entidades: los **clientes** y sus **pedidos**, los **pagos** de cada pedido (de los que se deriva el saldo), los **materiales** y **adjuntos** por pedido, las **facturas** emitidas al cliente y las **recibidas** de proveedores, los **usuarios** del sistema y las **suscripciones push** de cada dispositivo.

El modelo está **normalizado hasta la tercera forma normal (3FN)**. La normalización no es un trámite formal: es lo que evita que el mismo dato quede escrito en varios lugares y que esos lugares se contradigan con el tiempo. El apartado que sigue documenta qué se revisó y qué se corrigió.

Normalización hasta la tercera forma normal

Se partió de un modelo preliminar y se lo revisó forma por forma. En cada paso se buscó una dependencia funcional mal ubicada: un atributo que en realidad no depende de la clave de su tabla.

Forma	Qué exige	Qué se encontró	Cómo se resolvió
1FN	Cada celda guarda un único valor atómico y no hay grupos repetidos.	El campo `cliente.contacto` mezclaba en una sola celda teléfono y email (por ejemplo "11-5566-2233 / marta@mail.com"). El resto ya era atómico.	Conceptualmente se separa en `telefono` y `email`, los dos datos que la cobranza necesita por separado (el teléfono para WhatsApp, el email para el resumen).
2FN	1FN y además ningún atributo no clave depende solo de una parte de la clave.	Todas las tablas usan clave primaria simple (`id`), así que la 2FN se cumple por construcción. La verificación real es sobre la **clave candidata natural**: en `factura_cliente` esa clave es `numero`, y el `tipo` de comprobante no dependía del pedido sino del propio comprobante.	Se confirmó que `tipo`, `fecha` y `monto` dependen de la factura y no del pedido, y se mantuvieron en `factura_cliente` con `numero` como clave única.
3FN	2FN y además ningún atributo no clave depende de otro atributo no clave (sin dependencias transitivas).	Dos casos. El **saldo** de un pedido dependía de la suma de sus pagos, no del pedido en sí: guardarlo como columna lo haría quedar desactualizado en cuanto se cargara un pago. Y el **rol**, si se guardara como permisos sueltos en `usuario`, dependería del rol y no del usuario.	El saldo **no se guarda**: se calcula con la vista `v_saldo_pedido`. El rol quedó como un ENUM cerrado en `usuario`, y los permisos se resuelven en el backend (no hay tabla de permisos que mantener).

Dónde conviene no normalizar de más

No todo texto repetido es una violación de 3FN. `pedido.estado` (presupuestado, en_produccion, entregado, cerrado, no_concretado), `factura_cliente.tipo` (Factura A / B / Recibo) y `pago.medio` son dominios cerrados de pocos valores de los que no depende ningún otro atributo: se resuelven con un tipo `ENUM` y no necesitan tabla propia. Convertir cada enumeración en un catálogo agregaría *joins* sin ganar integridad.

En sentido contrario, **ningún saldo se almacena**: ni el del pedido, ni el del cliente, ni la deuda del proveedor. Los tres son datos **derivables** de los pagos y las facturas. Un valor derivable no viola la 3FN, pero duplica información y se puede desincronizar, así que se resuelven con vistas (`v_saldo_pedido`, `v_saldo_cliente`, `v_deuda_proveedor`). Es la razón del RNF07.

Tablas de hechos y tablas de dimensiones

Hecho y **dimensión** son términos del modelado dimensional. Espiga no es un modelo en estrella: es transaccional y está en 3FN. Lo que se identifica es el **rol que cumple cada tabla**, que es lo que hace legibles las consultas de reporte (las estadísticas de RF20 y el panel de saldos de RF02). Un **hecho** registra algo que pasó, tiene fecha y crece todo el tiempo; una **dimensión** describe el contexto, cambia poco y sirve para filtrar y agrupar.

Hecho	Grano (qué representa una fila)	Medida	Volumen
PAGO	Un pago de un cliente sobre un pedido, en una fecha.	`monto`, sumable. Es la base del saldo.	Alto. Crece con cada cobro.
FACTURA_CLIENTE	Un comprobante emitido por un pedido.	`monto`, sumable.	Medio.
FACTURA_PROVEEDOR	Una factura recibida de un proveedor.	`monto` y `pagado`; la deuda es su resta.	Medio.
PROVEEDOR_MOVIMIENTO	Una compra o un pago en una cuenta de proveedor.	`monto`, sumable con signo según el tipo. Es la base del saldo con proveedores.	Medio. Crece con cada compra y pago.
MATERIAL	Un material consumido en un pedido.	Ninguna numérica sumable (cantidad como texto).	Medio.

PAGO es el hecho principal, el que sostiene el saldo (RF03) y el cobrado por mes (RF20). Las tablas restantes son **dimensiones**, ordenadas por el eje de análisis que aportan:

Quién

CLIENTE y **USUARIO**: de quién es el pedido y quién cargó el pago.

Qué trabajo

PEDIDO, con **ADJUNTO** como detalle de sus archivos.

A quién se le compra

PROVEEDOR y **PROVEEDOR_CUENTA**: origen de las compras y cuenta en la que se registran.

A qué dispositivo

PUSH_SUSCRIPCION: dónde recibe las notificaciones cada usuario.

Cuándo: no hay tabla de tiempo. El tiempo vive en las columnas `fecha` de cada hecho. Un modelo en estrella agregaría una dimensión `tiempo` con año, mes y día de semana; acá el agrupamiento por mes de las estadísticas se resuelve con `DATE_FORMAT` en la consulta.

Se ve en las estadísticas (RF20). El "cobrado por mes" es el hecho PAGO agregado por la dimensión tiempo; los "mejores clientes" son PAGO agregado por la dimensión CLIENTE. Identificar los roles es lo que hace que esas consultas se lean solas.

Entidades

El núcleo del modelo tiene diez tablas. La **versión 1.0** sumó seis tablas y la **1.1** dos más (mismo criterio 3FN): **dieciocho en total**. Las nuevas se detallan a continuación y en el esquema relacional.

USUARIO

Personas con acceso (Rodrigo, Ana, Facu y quien se sume). El rol es un ENUM propio, no una tabla. Incluye email y datos de recuperación de clave.

CLIENTE

Clientes del taller, con su contacto. Eje de la cuenta corriente.

PROVEEDOR

Madereras y demás proveedores. Eje de las deudas.

PEDIDO

Un pedido de un cliente, con presupuesto, flete aparte, estado y fecha de entrega (exacta o en días hábiles).

PAGO hecho

Cada pago de un cliente sobre un pedido. De la resta contra el presupuesto sale el saldo. Es la tabla de mayor volumen.

MATERIAL

Consumo de materiales registrado en un pedido: descripción y cantidad.

ADJUNTO

Plano, foto o lista de corte de un pedido, guardado en el volumen persistente.

FACTURA_CLIENTE hecho

Comprobante que el taller emite (Factura A / B / Recibo) por un pedido.

FACTURA_PROVEEDOR hecho

Comprobante recibido de un proveedor, con lo pagado; la deuda es su resta.

PUSH_SUSCRIPCION

Dispositivo de un usuario suscripto a notificaciones push.

PEDIDO_ITEM v1.0

Renglones del presupuesto (descripción, cantidad, precio). El total del pedido sale de su suma.

PEDIDO_FOTO v1.0

Fotos del mueble; se ven en el seguimiento del cliente.

PEDIDO_ESTADO_HISTORIAL v1.0

Cada cambio de estado del pedido, con fecha y responsable.

INSUMO v1.0

Material del depósito con stock actual y mínimo.

INSUMO_MOVIMIENTO v1.0

Entradas y salidas de stock de un insumo.

PUSH_SEGUIMIENTO v1.0

Suscripción push del cliente a un pedido, desde su link.

PROVEEDOR_CUENTA v1.1

Cada cuenta de un proveedor. Todo proveedor arranca con una cuenta «General».

PROVEEDOR_MOVIMIENTO hecho v1.1

Compras y pagos de una cuenta. Su suma con signo da el saldo: le debés o a favor.

Relaciones

Entidad	Card.	Entidad	Significado
CLIENTE	1 : N	PEDIDO	Un cliente hace muchos pedidos.
PEDIDO	1 : N	PAGO	Un pedido recibe muchos pagos parciales.
PEDIDO	1 : N	MATERIAL	Un pedido consume varios materiales.
PEDIDO	1 : N	ADJUNTO	Un pedido tiene varios archivos adjuntos.
PEDIDO	1 : N	FACTURA_CLIENTE	Un pedido puede tener uno o más comprobantes emitidos.
PROVEEDOR	1 : N	FACTURA_PROVEEDOR	Un proveedor tiene muchas facturas recibidas.
USUARIO	1 : N	PAGO	Cada pago queda registrado con el usuario que lo cargó.
USUARIO	1 : N	FACTURA_CLIENTE	Cada comprobante queda registrado con quien lo emitió.
USUARIO	1 : N	PUSH_SUSCRIPCION	Un usuario puede tener notificaciones activas en varios dispositivos.
PEDIDO	1 : N	PEDIDO_ITEM	Un pedido se detalla en varios renglones. v1.0
PEDIDO	1 : N	PEDIDO_FOTO	Un pedido tiene varias fotos. v1.0
PEDIDO	1 : N	PEDIDO_ESTADO_HISTORIAL	Un pedido acumula sus cambios de estado. v1.0
INSUMO	1 : N	INSUMO_MOVIMIENTO	Un insumo acumula entradas y salidas. v1.0
PROVEEDOR	1 : N	PROVEEDOR_CUENTA	Un proveedor puede tener varias cuentas. v1.1
PROVEEDOR_CUENTA	1 : N	PROVEEDOR_MOVIMIENTO	Una cuenta acumula compras y pagos. v1.1

Esquema relacional (MySQL, tal como está implementado)

Definición de tablas (DDL) que materializa el modelo. Sirve como diccionario de datos. Las restricciones `UNIQUE` son las claves candidatas naturales usadas para verificar la 2FN. Los saldos aparecen como **vistas** al final, no como columnas.

```
-- ========== TABLAS PRINCIPALES ==========

CREATE TABLE usuario (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  nombre        VARCHAR(120) NOT NULL,
  email         VARCHAR(150),
  usuario_login VARCHAR(60)  NOT NULL UNIQUE,   -- clave candidata natural
  rol           ENUM('administrador','administrativo','operario') NOT NULL,
  hash_clave    VARCHAR(255) NOT NULL,          -- bcrypt, nunca texto plano (RNF01)
  activo        TINYINT(1) NOT NULL DEFAULT 1,
  reset_token   VARCHAR(64),                    -- recuperacion de clave (RF09)
  reset_expira  DATETIME
```

```
);

CREATE TABLE cliente (
  id        INT AUTO_INCREMENT PRIMARY KEY,
  nombre    VARCHAR(150) NOT NULL,
  contacto  VARCHAR(150),                      -- telefono y/o email
  activo    TINYINT(1) NOT NULL DEFAULT 1
);

CREATE TABLE proveedor (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  nombre  VARCHAR(150) NOT NULL,
  rubro   VARCHAR(100),
  activo  TINYINT(1) NOT NULL DEFAULT 1
);

CREATE TABLE pedido (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  cliente_id    INT NOT NULL REFERENCES cliente(id),
  descripcion   VARCHAR(255) NOT NULL,
  presupuesto   DECIMAL(12,2) NOT NULL DEFAULT 0,
  flete         DECIMAL(12,2) NOT NULL DEFAULT 0,      -- aparte del presupuesto (RF35)
  fecha_entrega DATE,
  dias_habiles  INT,                                   -- plazo en dias habiles (RF36)
  fecha_inicio_conteo DATE,
  estado        ENUM('presupuestado','en_produccion','entregado','cerrado','no_concretado')
                NOT NULL DEFAULT 'presupuestado'
);

-- Hecho principal: cada pago. El saldo se deriva de aca, no se guarda (3FN + RNF07)
CREATE TABLE pago (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  pedido_id   INT NOT NULL REFERENCES pedido(id),
  usuario_id  INT NOT NULL REFERENCES usuario(id),  -- trazabilidad (4.4.4)
  fecha       DATE NOT NULL,
  monto       DECIMAL(12,2) NOT NULL,
  medio       VARCHAR(60) NOT NULL
);

CREATE TABLE material (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  pedido_id   INT NOT NULL REFERENCES pedido(id),
  descripcion VARCHAR(150) NOT NULL,
  cantidad    VARCHAR(50) NOT NULL
);

CREATE TABLE adjunto (
  id             INT AUTO_INCREMENT PRIMARY KEY,
  pedido_id      INT NOT NULL REFERENCES pedido(id),
  nombre_archivo VARCHAR(200) NOT NULL,
  tipo           VARCHAR(60) NOT NULL,
  ruta           VARCHAR(255) NOT NULL           -- ubicacion en el volumen persistente
);

-- Comprobante emitido. numero es la clave candidata natural (unica)
CREATE TABLE factura_cliente (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  pedido_id   INT NOT NULL REFERENCES pedido(id),
  usuario_id  INT NOT NULL REFERENCES usuario(id),
  numero      VARCHAR(40) NOT NULL UNIQUE,
  fecha       DATE NOT NULL,
  monto       DECIMAL(12,2) NOT NULL,
  tipo        ENUM('Factura A','Factura B','Recibo') NOT NULL
);

-- Factura recibida. La deuda se deriva de (monto - pagado), no se guarda
```

```
CREATE TABLE factura_proveedor (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  proveedor_id  INT NOT NULL REFERENCES proveedor(id),
  usuario_id    INT NOT NULL REFERENCES usuario(id),
  numero        VARCHAR(40) NOT NULL,
  monto         DECIMAL(12,2) NOT NULL,
  fecha         DATE NOT NULL,
  pagado        DECIMAL(12,2) NOT NULL DEFAULT 0
);

-- Suscripcion push por dispositivo (RF17). endpoint es unico
CREATE TABLE push_suscripcion (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  usuario_id  INT NOT NULL REFERENCES usuario(id),
  endpoint    VARCHAR(500) NOT NULL UNIQUE,
  p256dh      VARCHAR(255) NOT NULL,
  auth        VARCHAR(255) NOT NULL
);

-- ========== TABLAS AGREGADAS EN LA VERSION 1.0 ==========
ALTER TABLE cliente  ADD COLUMN telefono VARCHAR(40);
ALTER TABLE pedido   ADD COLUMN token_seguimiento CHAR(20) UNIQUE;
ALTER TABLE material ADD COLUMN insumo_id INT REFERENCES insumo(id);

CREATE TABLE pedido_item (
  id INT AUTO_INCREMENT PRIMARY KEY, pedido_id INT NOT NULL REFERENCES pedido(id),
  descripcion VARCHAR(200) NOT NULL, cantidad DECIMAL(10,2) NOT NULL DEFAULT 1,
  precio_unitario DECIMAL(12,2) NOT NULL DEFAULT 0
);
CREATE TABLE pedido_foto (
  id INT AUTO_INCREMENT PRIMARY KEY, pedido_id INT NOT NULL REFERENCES pedido(id),
  ruta VARCHAR(255) NOT NULL, creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE pedido_estado_historial (
  id INT AUTO_INCREMENT PRIMARY KEY, pedido_id INT NOT NULL REFERENCES pedido(id),
  estado ENUM('presupuestado','en_produccion','entregado','cerrado','no_concretado') NOT NULL,
  usuario_id INT REFERENCES usuario(id), creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE insumo (
  id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(150) NOT NULL,
  unidad VARCHAR(30) NOT NULL DEFAULT 'unidad',
  stock_actual DECIMAL(12,2) NOT NULL DEFAULT 0, stock_minimo DECIMAL(12,2) NOT NULL DEFAULT 0,
  activo TINYINT(1) NOT NULL DEFAULT 1
);
CREATE TABLE insumo_movimiento (
  id INT AUTO_INCREMENT PRIMARY KEY, insumo_id INT NOT NULL REFERENCES insumo(id),
  cantidad DECIMAL(12,2) NOT NULL, motivo VARCHAR(150),
  usuario_id INT REFERENCES usuario(id), creado_en DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE push_seguimiento (
  id INT AUTO_INCREMENT PRIMARY KEY, pedido_id INT NOT NULL REFERENCES pedido(id),
  endpoint VARCHAR(500) NOT NULL UNIQUE, p256dh VARCHAR(255) NOT NULL, auth VARCHAR(255) NOT NULL
);

-- ========== VERSION 1.1 (cuentas por proveedor, RF34) ==========
CREATE TABLE proveedor_cuenta (
  id INT AUTO_INCREMENT PRIMARY KEY, proveedor_id INT NOT NULL REFERENCES proveedor(id),
  nombre VARCHAR(80) NOT NULL, activo TINYINT(1) NOT NULL DEFAULT 1
);
CREATE TABLE proveedor_movimiento (
  id INT AUTO_INCREMENT PRIMARY KEY, cuenta_id INT NOT NULL REFERENCES proveedor_cuenta(id),
  fecha DATE NOT NULL, tipo ENUM('cargo','pago') NOT NULL,   -- cargo = compra, pago = pago
  monto DECIMAL(12,2) NOT NULL, detalle VARCHAR(150), usuario_id INT REFERENCES usuario(id)
);

-- ========== VISTAS (datos derivados, no almacenados) ==========
```

```sql
-- Saldo de cada pedido = (presupuesto + flete) - suma de pagos
CREATE VIEW v_saldo_pedido AS
SELECT p.id AS pedido_id, p.cliente_id, p.presupuesto, p.flete,
       p.presupuesto + p.flete AS total_a_cobrar,
       COALESCE(SUM(pg.monto),0) AS total_pagado,
       p.presupuesto + p.flete - COALESCE(SUM(pg.monto),0) AS saldo
FROM pedido p LEFT JOIN pago pg ON pg.pedido_id = p.id
GROUP BY p.id, p.cliente_id, p.presupuesto, p.flete;

-- Saldo de cada cuenta de proveedor = compras - pagos (positivo: le debes; negativo: a favor)
CREATE VIEW v_deuda_proveedor_cuenta AS
SELECT cu.id AS cuenta_id, cu.proveedor_id, cu.nombre AS cuenta,
       COALESCE(SUM(CASE WHEN m.tipo = 'cargo' THEN m.monto ELSE -m.monto END),0) AS saldo
FROM proveedor_cuenta cu LEFT JOIN proveedor_movimiento m ON m.cuenta_id = cu.id
GROUP BY cu.id, cu.proveedor_id, cu.nombre;

-- Deuda con cada proveedor = suma de los saldos de sus cuentas
CREATE VIEW v_deuda_proveedor AS
SELECT pr.id AS proveedor_id, pr.nombre,
       COALESCE(SUM(CASE WHEN m.tipo = 'cargo' THEN m.monto ELSE -m.monto END),0) AS deuda
FROM proveedor pr
LEFT JOIN proveedor_cuenta cu ON cu.proveedor_id = pr.id AND cu.activo = 1
LEFT JOIN proveedor_movimiento m ON m.cuenta_id = cu.id
GROUP BY pr.id, pr.nombre;

-- Alertas de entrega (RF06): pedidos en produccion a 7 dias o menos
CREATE VIEW v_alertas_entrega AS
SELECT p.id AS pedido_id, p.descripcion, p.fecha_entrega, c.nombre AS cliente,
       DATEDIFF(p.fecha_entrega, CURDATE()) AS dias_restantes
FROM pedido p JOIN cliente c ON c.id = p.cliente_id
WHERE p.estado = 'en_produccion'
  AND DATEDIFF(p.fecha_entrega, CURDATE()) <= 7;
```

Lo que la normalización deja resuelto

El saldo (RF03) nunca se desincroniza. No es una columna que haya que actualizar con cada pago: es la vista `v_saldo_pedido`, que siempre refleja los pagos reales. Cargar un pago alcanza para que el saldo cambie en todas las pantallas.

El cierre del pedido es automático. Un *trigger* pasa el pedido a "cerrado" cuando está entregado y su saldo llegó a cero, sin que nadie tenga que hacerlo a mano — es la regla del diagrama de actividades del análisis, materializada en la base.

Un control que las claves foráneas no cubren. El número de una `factura_cliente` debe ser único a nivel de todo el taller: eso lo garantiza la restricción `UNIQUE` sobre `numero`, no una clave foránea.

5.7. Diagrama de flujo del sistema

Mientras el modelo entidad-relación (5.6) describe **qué guarda** el sistema, el diagrama de flujo describe **qué hace**: el recorrido de un pedido desde que se carga hasta que se cobra y se cierra, con las decisiones que el sistema evalúa. Está organizado por **carriles** (una columna por responsable), para leer también en qué capa ocurre cada paso.

ADMINISTRATIVO / ADMIN · Interfaz	BACKEND · Servidor	OPERARIO / USUARIO · Producción y celular
1. Ana o Rodrigo cargan un pedido para un cliente (o lo dan de alta si es nuevo), con presupuesto y fecha de entrega. (RF03, RF05)	**3.** El backend valida y guarda cada dato en MySQL, registrando el usuario que lo cargó. (ISW02, RNF06)	**4.** Facu actualiza el estado del pedido a medida que avanza la producción. (RF05)
2. Se adjuntan planos, fotos y la lista de corte; se registra el consumo de materiales. (RF07, RF14)	**5.** Una tarea diaria revisa las entregas: si un pedido en producción está a ≤7 días, arma la alerta. (RF17, ISW07)	**5b.** La alerta llega al celular como notificación push, aunque el sistema esté cerrado. (RF17)
6. Al cobrar, Ana carga el pago del pedido. (RF03)	**7.** El saldo se recalcula solo con la vista `v_saldo_pedido`. Si el pedido está entregado y el saldo llegó a 0, el trigger lo cierra. (RF03)	**8.** Rodrigo o Ana ven el pedido en el panel, el calendario y las alertas, y priorizan. (RF02, RF06, RF21)
9. Si corresponde, emite el comprobante (Factura A/B/Recibo) o el resumen en PDF. (RF12, RF13)	**10.** Cada noche genera el backup de los datos y conserva los últimos 7. (RF18, ISW09)	**11.** Si el cliente tiene saldo, Espiga arma el recordatorio listo para enviar por WhatsApp. (RF22)

Figura 1. Diagrama de flujo del sistema Espiga, con carriles por responsable. El ciclo de un pedido atraviesa las tres capas: la interfaz carga y consulta, el backend valida, calcula y avisa, y la producción actualiza el estado.

Qué decide el sistema y qué decide la persona. El sistema **calcula, guarda, avisa y respalda** de forma automática (saldos, alertas, cierre de pedidos, backups). Pero **el envío del recordatorio de cobranza lo confirma una persona**: Espiga arma el mensaje, Ana da el clic de enviar. El envío 100% automático está declarado como limitación (LIM05) y diferido a la v2 (4.6).

---

6. Oferta de Gestión

6.1. GANTT

Cronograma real del proyecto, del 20 de abril al 3 de noviembre de 2026, con defensa final la primera semana de noviembre. Las fechas, responsables y horas de cada tarea están en la planilla `Gantt_Espiga.xlsx`.

Fase	Fechas	Tareas principales	Responsable
1 · Análisis y mockup	20/04 – 10/05/2026	Análisis del taller, diagrama entidad-relación y de actividades, mockup navegable y matriz de permisos por rol.	Valentina Autelli · Mía Yorio (matriz de permisos: Felipe Araujo)
2 · Base de datos y backend	03/08 – 28/08/2026	Modelo MySQL en 3FN, vistas de saldo y triggers, API REST, autenticación y permisos por rol.	Felipe Araujo

Fase	Fechas	Tareas principales	Responsable
3 · Frontend conectado	28/08 – 11/09/2026	Conexión a la API, clientes y pagos, proveedores y pedidos, panel, alertas y usuarios.	Felipe Araujo
4 · Publicación	31/08 – 25/09/2026	Repositorio GitHub, base de datos en Railway, deploy y dominio, volumen persistente y pruebas en producción.	Felipe Araujo
5 · Mejoras, documentación y cierre	07/09 – 03/11/2026	Mejoras (RF12 a RF38), app instalable, rediseño, documentación, auditoría de cierre y simulación de defensa.	Felipe Araujo (mejoras) · Valentina Autelli y Mía Yorio (documentación) · Equipo (cierre)

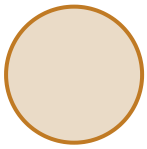
Planilla de cronograma. El detalle completo (tareas, responsables, fechas reales, % de cumplimiento y estado) se lleva en la planilla de cálculo del equipo, entregada junto a esta documentación como `Gantt_Espiga.xlsx`, con barras por semana y hitos por fase.

6.2. Infraestructura administrativa Buenas Prácticas UTN p.62

Organización interna del equipo y herramientas de trabajo. Siguiendo el esquema del libro UTN, el equipo se organiza en un grupo de dirección/gestión y un grupo de desarrollo, adaptado a la escala del proyecto de 7mo.

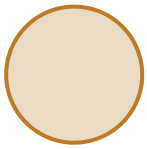
Grupo	Rol	Función
Dirección y Gestión	Director / PM	Cuestiones globales, coordinación, obtención de recursos y relación con el cliente.
	Líder de proyecto	Asignación de tareas, planificación temporal y económica.
Desarrollo	Backend / BD	Servidor, base de datos, API, despliegue en la nube.
	Frontend / UX-UI / Doc	Interfaz, mockup, validación con el usuario y documentación.

El equipo



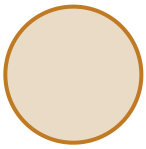
Felipe Araujo

Backend / BD · Frontend / UX-UI · Despliegue



Valentina Autelli

Project Manager



Mía Yorio

Documentación / QA

Área	Herramienta
Gestión de tareas	Planilla de cálculo: GANTT, asignación y estimación de costos.
Documentación	Este documento; código y README en GitHub.
Comunicación	Grupo de mensajería del equipo + reuniones de validación con el cliente.
Repositorio	GitHub (`felooaraujo-ai/Kore`); equipo docente como colaborador si es privado.
Despliegue	Railway, dominio `espiga-pcr.up.railway.app` , con volumen persistente.

---

7. Oferta Económica

Cotización de los servicios e insumos necesarios para el funcionamiento del sistema. Al ser un proyecto 100% software, no hay insumos electrónicos ni de fabricación: los únicos costos recurrentes son de infraestructura en la nube.

7.1. Infraestructura y hosting

Ítem	Detalle	Costo
Hosting, base de datos y almacenamiento	Railway: backend + frontend, MySQL y volumen persistente para archivos y backups.	2,50 USD / mes
Asistente de desarrollo con IA	Claude, usado durante el desarrollo y la documentación.	20,00 USD / mes
Envío de emails	Recuperación de clave y backups por mail (Brevo).	Sin costo
Dominio propio (opcional)	No incluido: se usa el subdominio `espiga-pcr.up.railway.app` .	No incluido
Total mensual	Costo externo mientras dura el desarrollo.	**22,50 USD / mes**

> El estudio ya cuenta con PC, celulares e internet, por lo que no se cotiza el equipamiento del cliente. El software es todo de código abierto, sin costo de licencia.

---

8. Estimación Económica

Presupuesto total del proyecto. Suma el costo de infraestructura (sección 7) más el esfuerzo en **horas hombre**, diferenciado por categoría profesional y su costo por hora (transparencia en el esfuerzo, según los Lineamientos 2026).

8.1. Horas hombre por categoría

Estimación de esfuerzo por fase y por categoría. Las horas son una estimación del equipo para las 28 semanas del cronograma; el costo de referencia es de 5 USD por hora hombre.

Categoría	Tareas	Horas estim.	$/hora	Subtotal
Análisis y diseño	Relevamiento, ERS, modelo de datos, normalización.	40 h	5 USD	200 USD
Backend / Base de datos	API, lógica, vistas, triggers, seguimiento, stock, ítems, push, mails, PDF/QR, seguridad.	176 h	5 USD	880 USD
Frontend / UX-UI	Interfaz, API, rediseño, PWA, modo oscuro, Kanban, dashboard, galería.	168 h	5 USD	840 USD
QA / Pruebas	Pruebas, debugging, verificación en producción.	40 h	5 USD	200 USD
Documentación	Manual de usuario, técnico, esta carpeta de proyecto.	40 h	5 USD	200 USD
Gestión (PM)	Coordinación, relación con el cliente, planificación.	40 h	5 USD	200 USD
	Total horas	**508 h**	5 USD	**2.540 USD**

8.2. Presupuesto total

Rubro	Detalle	Monto
Horas hombre	508 h × 5 USD (8.1).	2.540,00 USD
Asistente de IA (Claude)	20 USD/mes × 4 meses de desarrollo (agosto a noviembre).	80,00 USD
Infraestructura (Railway)	2,50 USD/mes × 3 meses publicados (septiembre a noviembre).	7,50 USD
Total del proyecto	Suma de los rubros anteriores.	**2.627,50 USD**
Costo de mantenimiento	Después de la entrega solo queda Railway.	2,50 USD / mes

---

9. Contrato

Documento que explicita el acuerdo entre el equipo desarrollador y el cliente (Rodrigo Pasut, PCR Amoblamientos). Se firma en la reunión de validación, habiendo leído las limitaciones (5.1).

9.1. Objetivo del proyecto

Desarrollar, implementar y poner en marcha Espiga, un sistema de gestión web que centraliza clientes con cuenta corriente, proveedores, pedidos, archivos técnicos, cobranzas y entregas, con acceso diferenciado por rol, para eliminar la gestión manual y dispersa del taller.

9.2. Partes involucradas

Equipo desarrollador (integrantes y roles según 3.3) y cliente: Rodrigo Pasut, titular de PCR Amoblamientos (Villa Bosch).

9.3. Alcance

Se entrega lo definido en 3.2 y 5: login con 3 roles, panel de saldos, clientes, proveedores, pedidos, archivos, consumo de materiales, comprobantes y presupuestos en PDF, alertas de entrega (con notificación al celular), calendario, estadísticas, recordatorios de cobranza, backup automático, gestión de usuarios y app instalable, todo publicado en Railway. Queda fuera lo listado en las Limitaciones (LIM01 a LIM05).

9.4. Plazos

Fechas clave del cronograma (6.1): análisis y mockup del 20/04 al 10/05/2026; base de datos y backend del 03/08 al 28/08; frontend conectado del 28/08 al 11/09; publicación del 31/08 al 25/09; mejoras, documentación y cierre del 07/09 al 03/11/2026. Entrega y defensa final: primera semana de noviembre de 2026.

9.5. Responsabilidades

El equipo: desarrollo, despliegue, puesta en marcha y documentación. El cliente: validar requerimientos y prototipo, y proveer su email real para activar la recuperación de contraseña.

9.6. Criterios de aceptación

Se apoyan en los OKRs: saldo de clientes y proveedores calculado automáticamente, alertas de entrega generadas para todo pedido a ≤7 días, archivos conservados entre reinicios, backup diario funcionando, y el sistema accesible desde cualquier dispositivo.

9.7. Condiciones de modificación

Todo cambio de alcance solicitado durante el desarrollo se registra en el control de cambios (sección 1) y se evalúa su impacto en el GANTT y en la oferta económica antes de aceptarse.

9.8. Firmas

Firma del equipo desarrollador y de Rodrigo Pasut (simbólica, en la validación del prototipo).

---

10. Bitácora del Proyecto

Documento de trabajo del equipo. Registra la investigación de campo (entrevistas y minutas) y el seguimiento de la iteración (cambios de interfaz y avances técnicos por fase). No describe el sistema sino el proceso, y se actualiza a medida que el proyecto avanza.

10.1. Registro de Entrevistas

Fecha	Con quién	Objetivo	Modalidad
20/04/2026	**Rodrigo Pasut** · titular del taller	**Entrevista 1.** Identificación del problema real y relevamiento de la línea de base (cómo se lleva hoy el saldo, los pedidos y las deudas).	Presencial
24/04/2026	**Rodrigo Pasut**	**Entrevista 2.** Detalle de los tres roles del taller (quién hace qué) y del circuito de un pedido, del presupuesto al cobro.	Presencial
10/05/2026	**Rodrigo Pasut**	**Validación del prototipo.** Feedback sobre el mockup antes de conectar la lógica real.	Presencial / Virtual
05/10/2026	**Rodrigo Pasut**	**Pedidos de mejora.** Cuentas separadas por proveedor, flete aparte del presupuesto y entregas en días hábiles con aviso.	Presencial

10.2. Minutas de Reunión

Acta breve de cada reunión, siempre con los mismos seis campos, para que cualquier participante pueda reconstruir qué se decidió y quién quedó a cargo de qué.

Minuta 01 · Validación del prototipo con el cliente	
Fecha y duración	10/05/2026. Duración estimada: 45 minutos.
Participantes	Rodrigo Pasut (cliente) y el equipo de proyecto.
Objetivo	Validar el mockup navegable con Rodrigo antes de conectar la lógica real (5.2.1).
Temas tratados	Recorrido de las pantallas (panel de saldos, clientes, pedidos, alertas); lectura de la tabla de Limitaciones (5.1); prueba en el celular.
Decisiones y acuerdos	No mostrar usuarios ni claves de ejemplo en el login; sumar la sección «¿Cómo funciona?»; confirmar los permisos de cada rol. Las limitaciones se aceptan como están declaradas.
Próximos pasos	El equipo aplica los cambios y los asienta en el Registro de Cambios de Interfaz (10.3). El PM abre la fila en el Control de Cambios (sección 1).

Minuta 02 · Pedidos de mejora del cliente	
Fecha y duración	05/10/2026. Duración estimada: 30 minutos.
Participantes	Rodrigo Pasut (cliente) y Felipe Araujo.
Objetivo	Relevar ajustes del sistema en uso real en el taller.
Temas tratados	Saldos de proveedores mezclados en el panel, necesidad de separar cuentas de un mismo proveedor, flete aparte del presupuesto y plazos de entrega en días hábiles.

Decisiones y acuerdos	Cada proveedor por separado y con varias cuentas; flete como línea aparte que se suma a cobrar; entrega en días hábiles salteando feriados, con aviso unos días antes y el día.
Próximos pasos	Implementar RF34 a RF37 y registrarlos en el Control de Cambios (versión 1.1).

10.3. Registro de Cambios de Interfaz

Modificaciones aplicadas a la interfaz después de validarla con Rodrigo. Es la trazabilidad de la iteración con el usuario.

Componente modificado	Qué se modificó	Por qué (justificación UX / usuario)
Pantalla de login	Se quitaron los usuarios y claves de ejemplo que se mostraban al pie.	Era un hueco de seguridad: cualquiera que abriera la página veía credenciales válidas (RNF02).
Navegación en celular	Barra inferior simplificada (Inicio, Clientes, Prov., Pedidos, Alertas) distinta del menú de escritorio.	En pantalla chica, la barra lateral completa no entra; se prioriza lo que se usa en la calle.
Estética general	Rediseño con identidad "madera" (login, barra, menú, tarjetas), marcando el ítem activo del menú.	Un taller de muebles se identifica más con una estética cálida de madera que con un panel gris genérico.

10.4. Bitácora de Avances Técnicos

Hitos técnicos cerrados, en orden, siguiendo las fases del cronograma (6.1).

Fase	Hitos registrados	Fecha de cierre
1–2	Documento de análisis: mockup, roles, diagrama entidad-relación y de actividades. Modelo de datos normalizado a 3FN.	21/08/2026
2–3	Base de datos y backend construidos paso a paso, cubriendo RF01 a RF11. Conexión del mockup a la API real.	11/09/2026
4	Publicación en Railway (repo `felooaraujo-ai/Kore`), subida real de archivos con volumen persistente y dominio propio.	11/09/2026
5	Recuperación de clave por email (pausada), comprobantes y PDF, consumo de materiales, buscador y filtros (RF12–RF15).	18/09/2026
5	App instalable (PWA) y notificaciones push probadas en un iPhone real (RF16, RF17).	18/09/2026
5	Backup automático, presupuesto en PDF, estadísticas, calendario y recordatorios de cobranza (RF18–RF22). Rediseño estético.	02/10/2026
5	Seguimiento del cliente, presupuesto por ítems con QR, Excel, historial, stock de insumos, Kanban, fotos, dashboard y flujo de caja (RF23–RF33). Recuperación de clave activa por email.	02/10/2026

Fase	Hitos registrados	Fecha de cierre
5	Cuentas por proveedor, flete aparte, entrega en días hábiles con feriados y panel por proveedor (RF34–RF37).	05/10/2026
5	Puesta en marcha con datos reales (RF38). Manual y preguntas frecuentes actualizados.	21/10/2026
5	Documentación publicada en PDF, identidad corporativa y pieza visual para la Expo.	05/11/2026

Criterio de cierre. Un hito se da por cerrado cuando su entregable está verificado en producción, no cuando se termina de trabajar en él.

---

11. Guía de Inicio Rápido

Puesta en marcha para Rodrigo y su equipo. Sin jerga técnica: no hace falta saber qué es una API ni una base de datos para usar el sistema.

Antes de empezar. Solo hacen falta dos cosas: un navegador en la PC o el celular, y el usuario y la contraseña que entrega el administrador. No se instala nada (aunque opcionalmente se puede agregar como app en el celular). La app de la PC y la del celular son la misma: todo lo que se actualiza aparece solo en los dos.

Primeros pasos

1. **Entrar al sistema.** Abrir `espiga-pcr.up.railway.app` e ingresar con usuario y clave. Después de 5 intentos fallidos el ingreso se bloquea unos minutos, por seguridad. (RF01)
2. **Puesta en marcha (solo una vez).** El administrador entra a **Usuarios → Preparar para el cliente** y escribe `BORRAR EJEMPLOS`. Se quitan los clientes, proveedores, pedidos y usuarios de muestra que trae el sistema de fábrica; lo cargado desde la app no se toca y antes se envía un backup por mail. Después, cambiar la clave del administrador desde **Mi cuenta**.
3. **Cargar al equipo.** En **Usuarios → Nuevo usuario**, crear a cada persona con su rol (administrativo u operario). La clave debe tener 6 o más caracteres, con letras y números. (RF08)

Uso diario

1. **Revisar el panel.** Muestra cuánto te deben los clientes, el flujo de caja de los últimos 6 meses, los clientes con saldo, el saldo con cada proveedor (por separado, sin mezclarlos en un total) y las próximas entregas. (RF02, RF33)
2. **Crear un cliente con su pedido.** Nombre (obligatorio, marcado con *), teléfono, email, descripción y presupuesto. Los montos se escriben solo con números y se formatean solos ($ 310.000). (RF03, RF05)

3. **Cargar el flete.** El flete va **aparte del presupuesto**: se puede cargar al crear el pedido o el día de la entrega desde la ficha del pedido (*Cargar flete*). Se suma a lo que paga el cliente como línea separada: Presupuesto + Flete = Total a cobrar.

4. **Fijar la entrega.** Elegir **Días hábiles** y escribir cuántos (los que necesite el proyecto, por ejemplo 45) o **Fecha exacta**. Con días hábiles el sistema calcula la fecha salteando fines de semana y feriados nacionales, y la muestra en el momento. Se puede cambiar después con *Editar entrega*.

5. **Detallar el presupuesto por ítems** (opcional): en la ficha del pedido se agregan renglones con cantidad y precio y el total se calcula solo. El presupuesto en PDF incluye los ítems, el flete y un código QR al seguimiento. (RF19, RF24, RF25)

6. **Registrar un pago** al cobrar (efectivo, transferencia o cheque). El saldo se recalcula solo. Se puede emitir el comprobante (Factura A, B o Recibo) y el resumen de cuenta en PDF. (RF03, RF12, RF13)

7. **Compartir el seguimiento con el cliente.** En la ficha del pedido, *Copiar link de seguimiento* y mandarlo por WhatsApp. El cliente ve, sin usuario ni clave, el estado, la entrega estimada, las fotos y su saldo, y puede activar avisos cuando avanza. (RF23, RF28)

8. **Seguir la producción.** En **Producción**, las tarjetas de los pedidos se arrastran entre columnas (Presupuestado → En producción → Entregado). Cada cambio queda en el historial del pedido con fecha y usuario. Al marcar entregado un pedido sin saldo, se cierra solo. (RF27, RF30)

9. **Fotos, planos y materiales.** Subir fotos del mueble a la galería (el cliente también las ve), adjuntar planos o listas de corte (solo PDF, JPG o PNG) y registrar el material usado. Si el material se elige del stock, se descuenta solo de Insumos. (RF07, RF14, RF29, RF31)

10. **Manejar proveedores.** Cada proveedor muestra su propio saldo: **Le debés** o **A favor**. Dentro de cada proveedor se pueden crear varias **cuentas** (por ejemplo, para separar compras por tipo de material o forma de pago) con *Nueva cuenta*; cada una tiene su saldo y sus movimientos: *+ Compra / le debés* y *− Registrar pago*. (RF04)

11. **Controlar el stock.** En **Insumos** se carga cada material con su stock y mínimo, y se registran entradas y salidas. Avisa cuando algo está por agotarse. (RF29)

12. **Revisar alertas y calendario.** El calendario muestra todos los pedidos por fecha de entrega. Desde 7 días antes y el mismo día de la entrega, el sistema avisa (también al celular, si están activadas las notificaciones). (RF06, RF21)

13. **Recordatorios de cobro.** Lista de clientes con saldo y un mensaje ya redactado para copiar o abrir en WhatsApp. (RF22)

14. **Estadísticas y exportar.** Gráficos de cobros por mes, mejores clientes y proveedores con más compras. Clientes, pedidos y pagos se pueden descargar en Excel. (RF26, RF32)

15. **Backups.** El sistema hace una copia diaria (guarda las últimas 7) y una semanal por mail; el administrador puede descargarla o enviarla en el momento desde **Usuarios → Copias de seguridad**.

(RF18, RF33)

16 **Celular y modo oscuro.** Agregar Espiga a la pantalla de inicio y activar las notificaciones de entrega desde **Mi cuenta**. El botón de la luna cambia a modo oscuro. (RF16, RF17, RF33)

---

12. Preguntas Frecuentes

Las preguntas que el cliente y los usuarios del taller hacen con más frecuencia, con la respuesta y la referencia al requerimiento o la limitación que la respalda.

¿Hay que instalar algo para usar Espiga?

No. Se usa desde el navegador, en la PC o en el celular, entrando a `espiga-pcr.up.railway.app`. No depende de ninguna computadora del taller (RF11). Opcionalmente se puede instalar como app en el celular (RF16).

¿El sistema calcula el saldo de los clientes solo?

Sí. Cada pago que se carga actualiza automáticamente el saldo; no hace falta sumar a mano (RF03). El saldo nunca se guarda como un número fijo: se calcula en el momento a partir de los pagos, para que no pueda quedar desactualizado (RNF07).

¿Rodrigo, Ana y Facu ven lo mismo al entrar?

No. Cada uno tiene un rol distinto (administrador, administrativo, operario) y ve solo lo que necesita para su tarea (RF01). Además, cada pago queda registrado con el usuario que lo cargó (4.4.4).

¿Qué pasa si me olvido la clave?

En el login tocás «¿Olvidaste tu clave?» y te llega un link por email para crear una nueva. La recuperación está **activa** mediante la API de Brevo (RF09). El administrador también puede resetear claves desde Usuarios.

¿El cliente puede ver cómo va su pedido?

Sí. Cada pedido tiene un **link de seguimiento** público (sin usuario ni clave) donde el cliente ve el estado, la entrega estimada, las fotos y su pago. Puede activar avisos para que le llegue una notificación cuando avanza (RF23, RF28, RF31).

¿Cómo controlo el stock de materiales?

En la sección **Insumos**: cargás cada material con su stock y mínimo, registrás entradas y salidas, y cuando consumís un insumo en un pedido se descuenta solo. Te avisa cuando algo está por agotarse (RF29).

¿Qué archivos puedo adjuntar a un pedido?

PDF, JPG o PNG (hasta 25 MB). Para el avance del mueble se usa la galería de fotos del pedido (RF07, RF31).

¿Dónde quedan guardados los planos y las listas de corte?

En un almacenamiento persistente vinculado al pedido. No se pierden si el sistema se reinicia o se actualiza (RF07, RNF05).

¿El sistema emite factura o se conecta con AFIP?

Emite comprobantes simples por pedido (Factura A, Factura B o Recibo), pero no factura electrónica ni se integra con AFIP en esta entrega (RF12, LIM01). También genera presupuestos y resúmenes de cuenta en PDF (RF13, RF19).

¿Se puede usar desde el celular?

Sí. Además de la interfaz simplificada, el sistema se puede instalar como un ícono en la pantalla de inicio (Android e iPhone) y, una vez instalado, avisa solo al celular cuando un pedido está por vencerse (RF16, RF17). En iPhone, las notificaciones requieren tenerlo instalado como app (es una restricción de Apple).

¿Se pierden los datos si algo falla?

No. El sistema genera un backup automático de todos los datos todos los días y conserva los últimos siete, que el administrador puede descargar cuando quiera (RF18).

¿Cómo sé qué clientes me deben?

La sección Recordatorios lista los clientes con saldo pendiente, ordenados, con el total por cobrar y un mensaje ya redactado listo para copiar o abrir en WhatsApp (RF22).

¿Puedo separar las cuentas de un mismo proveedor?

Sí. Dentro de cada proveedor se crean las cuentas que hagan falta con *Nueva cuenta*. Cada cuenta tiene su propio saldo (**Le debés** o **A favor**) y sus movimientos, y se ven discriminadas al abrir el proveedor. Los saldos de distintos proveedores no se mezclan en un total.

¿El flete se suma al presupuesto?

No se mezcla: queda como línea aparte. Se puede cargar el día de la entrega y se suma a lo que paga el cliente (Presupuesto + Flete = Total a cobrar). Aparece en la ficha, en el presupuesto en PDF y en el link de seguimiento.

¿Puedo poner la entrega en días hábiles?

Sí. Al crear o editar un pedido elegís **Días hábiles** y escribís la cantidad que necesite el proyecto. El sistema calcula la fecha salteando sábados, domingos y feriados nacionales, y avisa desde 7 días antes y el mismo día de la entrega.

¿Cómo se sacan los datos de ejemplo para empezar a usarlo en serio?

El administrador va a **Usuarios → Preparar para el cliente** y confirma escribiendo `BORRAR EJEMPLOS`. Se borran solo los datos de muestra de fábrica; lo cargado desde la app se conserva y antes se manda un backup por mail. Después conviene cambiar la clave del administrador.

¿El sistema guarda datos sensibles?

Guarda datos comerciales del taller (clientes, pedidos, pagos), no datos personales sensibles. Las contraseñas se almacenan con hash y nunca en texto plano, y la base de datos no tiene acceso público: solo el backend le habla (4.4.8).

Cómo se mantiene. Cada pregunta nueva que surge en una reunión con el cliente se suma acá y se anota en la minuta correspondiente (10.2). Si la respuesta obliga a cambiar el alcance, se registra en el Control de Cambios (sección 1).

Espiga · Documentación del Proyecto · PCR Amoblamientos, Villa Bosch.
Evaluación de Proyectos · 7mo Informática 2026 · Instituto Leonardo Murialdo.
Equipo: Felipe Araujo · Valentina Autelli · Mía Yorio.